

컴퓨터공학 종합설계 프로젝트 (CAPSTONE)

SkyOps Intelligence 최종 보고서

실시간 항공 이상 탐지 및 AI 어드바이저 코파일럿 플랫폼

발표일: 2026년 4월 16일

팀: DoubleJ 팀 (정재원 외)

버전: v2.1.3

본 보고서는 초보자도 프로젝트 구조와 기술 선택 이유를 이해할 수 있도록 작성되었습니다.

1. 요약 (Executive Summary)

SkyOps Intelligence는 실시간으로 항공 데이터를 수집하고, 이상을 탐지하며, 운항 관제사에게 AI 기반 조언을 제공하는 통합 플랫폼입니다. 본 프로젝트는 대학 캡스톤 수준을 넘어 엔터프라이즈급 운영성을 확보하는 것을 목표로 하여, 데이터 파이프라인·머신러닝 모델·대규모 언어 모델(LLM)·서빙 API·모니터링·배포까지 전체 스택을 자체 구축하였습니다.

핵심 성과

- ADS-B, METAR, SWIM 등 실시간 항공 데이터를 분당 수천 건 수집·처리하는 Kafka 스트림 파이프라인 구축
- XGBoost + Conformal Prediction으로 지연 시간 예측 시 90% 커버리지 신뢰구간 제공
- Per-phase Isolation Forest 7개를 운항 단계(이륙/순항/착륙 등)별로 학습하여 False Positive 대폭 감소
- Qwen2.5-7B 모델을 QLoRA + DPO로 파인튜닝하고, ChromaDB RAG와 결합하여 항공 규정 기반 조언 생성
- FastAPI 서빙 15개 엔드포인트, Next.js 16 대시보드, Docker Compose + Helm Chart 배포
- OpenTelemetry + Prometheus + Grafana + OpenLineage로 전 구간 관측성(Observability) 확보

기술적 차별점

본 프로젝트의 가장 큰 차별점은 '왜 이 기술을 썼는지'가 명확하다는 점입니다. 단순히 유행을 따르지 않고, 항공 도메인의 특수성(안전성, 실시간성, 낮은 False Positive)을 고려하여 각 컴포넌트를 선택하였습니다. 예를 들어 XGBoost는 결측치에 강하고 해석 가능성이 높기 때문에 선택하였고, Conformal Prediction은 분포 가정 없이 통계적 보장을 제공하므로 안전 민감 도메인에 적합합니다.

2. 목차

1. 요약 (Executive Summary)
2. 프로젝트 개요와 배경
3. 시스템 아키텍처
4. 데이터 수집과 스트림 파이프라인
5. 머신러닝 모델 (지연 예측)
6. 이상 탐지 모델
7. LLM과 RAG
8. 서빙 API와 대시보드
9. MLOps와 관측성
0. 배포 전략
1. 성능 평가
2. 결론 및 향후 과제
3. 참고문헌
4. 부록 A · API 엔드포인트 명세
5. 부록 B · 배포 체크리스트
6. 부록 C · 용어집

3. 프로젝트 개요와 배경

3.1 문제 정의

항공 산업은 매일 전 세계적으로 10만 편 이상의 항공기가 운항하며, 날씨·교통량·정비·인적 오류 등 다양한 요인으로 지연과 이상 상황이 발생합니다. 현재 운항 관제사와 항공사 운영센터(Operations Center)는 여러 시스템을 동시에 모니터링하며 수동으로 판단해야 합니다. 이는 의사결정 속도를 늦추고 인적 실수를 유발할 수 있습니다.

3.2 해결하고자 하는 것

- 실시간 항공 데이터(위치, 속도, 고도, 기상, 관제 메시지)를 한 곳에 통합
- 머신러닝으로 지연을 사전 예측하고, 불확실성(신뢰구간) 제공
- 이상 행동(예: 경로 이탈, 급격한 고도 변화)을 자동으로 탐지
- 탐지 결과에 대해 LLM이 항공 규정(ICAO, FAA AIM)을 참고하여 조치 방안 제안
- 관제사가 한눈에 볼 수 있는 대시보드 제공

3.3 프로젝트 범위

본 프로젝트는 '데이터 수집부터 운영 배포까지' 엔드투엔드(End-to-End)로 구현합니다. 이는 단순히 모델 하나를 학습하는 것을 넘어, 데이터 엔지니어링·ML·MLOps·LLMOps·DevOps가 유기적으로 연결된 시스템입니다.

3.4 왜 이 문제가 중요한가

항공 지연 1분은 항공사에게 약 \$65의 비용을 발생시키고(IATA 추정), 연간 전 세계적으로 수십억 달러의 손실을 만듭니다. 또한 이상 상황이 조기에 탐지되지 않으면 사고로 이어질 수 있어 인명 손실의 위험도 있습니다. AI·ML 기반 조기 경보와 의사결정 지원은 이러한 리스크를 줄이는 핵심 기술입니다.

4. 시스템 아키텍처

4.1 전체 구조

시스템은 크게 6개 레이어로 구성됩니다: (1) 데이터 수집, (2) 스트림 처리, (3) 저장소, (4) 머신러닝·LLM 서빙, (5) 대시보드, (6) 관측성·MLOps. 각 레이어는 독립적으로 확장 가능하며, Kubernetes 기반으로 마이크로서비스로 배포됩니다.

4.2 레이어별 역할

레이어	주요 컴포넌트	역할
데이터 수집	OpenSky ADS-B, METAR, FAA SWIM	항공기 위치·기상·관제 메시지 실시간 수집
스트림 처리	Kafka + Avro + PyFlink	이벤트 스키마 검증 후 실시간 변환·저장
저장소	Apache Iceberg (Bronze/Silver/Gold), Redis, ChromaDB	원본 데이터·피처·벡터 저장
ML·LLM 서빙	FastAPI + vLLM + XGBoost + Isolation Forest	예측·이상탐지·LLM 조언 생성
대시보드	Next.js 16 App Router	실시간 지도·통계·이상 알림 UI
관측성·MLOps	OpenTelemetry + Prometheus + Grafana + MLflow	추적·메트릭·모델 버전 관리

4.3 왜 마이크로서비스로 분해했는가

단일 모놀리식 애플리케이션이면 배포가 간단하지만, 장애 격리·독립 확장·팀별 개발이 어렵습니다. 마이크로서비스로 분해하면 LLM 서빙(GPU 필요)과 데이터 수집(CPU만 필요)을 서로 다른 노드에 배치할 수 있어 리소스 효율이 높습니다. 또한 LLM 서비스만 장애가 나도 자연 예측 API는 계속 동작합니다.

4.4 핵심 데이터 흐름

- OpenSky 프로듀서가 ADS-B 위치 데이터를 Kafka 'flight-position' 토픽에 발행
- PyFlink가 스트림을 받아 피처 엔지니어링 후 Iceberg Silver 테이블에 저장
- 이상 탐지 워커가 Silver 데이터를 읽어 Isolation Forest로 점수 계산, 이상 시 Redis에 저장
- 대시보드가 SSE(Server-Sent Events)로 Redis 스트림을 구독하여 실시간 알림
- 사용자가 '왜 이상인지?' 클릭 시 LLM이 RAG로 ICAO 문서 참조하여 설명 생성

5. 데이터 수집과 스트림 파이프라인

5.1 데이터 소스

본 프로젝트는 3가지 공개·표준 항공 데이터 소스를 통합합니다. 각 소스는 서로 다른 포맷·주기·신뢰성을 가지므로, 통일된 스키마로 정규화하는 것이 핵심입니다.

소스	포맷	주기	무엇을 알려주나
OpenSky Network (ADS-B)	JSON	10초	항공기 위치·속도·고도·Heading
NOAA METAR	텍스트	30분	공항 시정·풍향·기온·기압
FAA SWIM (SFDPS, TFMS)	AIXM 5.1 XML / FAA event: namespace	이벤트	미국 영공 NOTAM, 교통 흐름 제어(TFM), ATFM 제한

5.2 왜 Kafka를 썼는가

항공 데이터는 초당 수십~수백 건의 이벤트가 발생하고, 여러 소비자(ML 파이프라인, 대시보드, 모니터링)가 동시에 읽어야 합니다. Kafka는 분산 커밋 로그로 다음 장점을 제공합니다:

- 높은 처리량: 초당 수백만 메시지도 처리 가능
- 내구성: 이벤트가 디스크에 저장되어 소비자가 재시작해도 재처리 가능
- 분리(Decoupling): 생산자와 소비자가 서로 몰라도 됨 → 독립 배포
- 순서 보장: 파티션 단위로 이벤트 순서 유지

예시: OpenSky에서 읽어온 위치 데이터를 Kafka에 쓰면, 대시보드·ML 학습·이상 탐지 3개의 소비자가 동시에 소비할 수 있습니다. 각자 자기 속도로 읽기 때문에 느린 소비자가 빠른 소비자를 막지 않습니다.

5.3 왜 Avro Schema Registry인가

JSON은 스키마가 없어 생산자가 필드를 추가·변경하면 소비자가 깨집니다. Avro는 바이너리 포맷이면서 스키마를 별도로 등록하여 스키마 진화(Schema Evolution)를 안전하게 관리합니다. Confluent Schema Registry가 이를 중앙에서 관리하므로, 모든 토픽의 스키마 호환성(Backward/Forward)을 강제할 수 있습니다.

5.4 왜 PyFlink인가

Flink는 진정한 스트림 처리 엔진으로 Event-time 기반 윈도우, 정확히 한 번(Exactly-Once) 시맨틱을 제공합니다. Spark Streaming은 마이크로배치 기반이라 지연이 있지만, Flink는 진짜 스트림이므로 실시간 이상 탐지에 더 적합합니다. PyFlink를 쓴 이유는 팀이 Python에 익숙하고, ML 파이프라인과 쉽게 통합할 수 있기 때문입니다.

5.5 Medallion Architecture (Bronze/Silver/Gold)

저장소는 3층 구조로 분리합니다:

- Bronze (원본): Kafka에서 받은 그대로 저장. 디버깅·재처리에 사용.
- Silver (정제): 스키마 검증·결측치 처리·표준화 완료. ML 학습에 사용.
- Gold (집계): 피처·KPI 테이블. 대시보드·리포트에 사용.

이 구조의 장점은 '어떤 가공 단계에서 문제가 생겼는지' 추적이 쉽고, Silver에서 재처리하면 Gold를 다시 만들 수 있어 실수를 되돌리기 쉽습니다. Apache Iceberg는 이 저장장을 ACID 트랜잭션과 타임트래블(Time Travel)로 지원합니다.

다.

6. 머신러닝 모델 — 지연 예측

6.1 문제 정의

목표: 항공기 한 편이 주어진 피처(출발지, 도착지, 시간대, 기상, 항공사, 기종 등)를 가질 때, 예상 지연 시간(분)을 예측하고, 신뢰구간을 함께 제공한다.

6.2 왜 XGBoost인가

많은 후보 모델 중에서 XGBoost(eXtreme Gradient Boosting)를 선택한 이유는 다음과 같습니다:

기준	XGBoost	Neural Network	Random Forest
표 형식 데이터 성능	★★★★★	★★★	★★★★★
결측치 처리	자동 지원	별도 imputation 필요	제한적
해석 가능성 (SHAP)	★★★★★	★★	★★★★★
학습 속도	빠름	느림 (GPU 필요)	중간
과적합 방어	Regularization 내장	Dropout 필요	Bagging

항공 도메인에서는 '왜 이 예측이 나왔는지' 설명할 수 있어야 합니다. 규제 기관이나 파일럿이 AI의 판단을 신뢰하려면 해석 가능해야 하기 때문입니다. SHAP(SHapley Additive exPlanations)와 결합된 XGBoost는 각 피처의 기여도를 정확히 계산할 수 있어 이 요구에 부합합니다.

6.3 Conformal Prediction — 왜 필요한가

단순히 '이 항공편은 15분 지연'이라고 말하는 것은 위험합니다. 모델이 얼마나 확신하는지 알 수 없기 때문입니다. Conformal Prediction은 분포 가정 없이(distribution-free) 통계적 보장을 제공하는 기법으로, 다음을 보장합니다: '이 구간 [12분, 18분]에 실제 값이 들어올 확률은 90%다.'

- Split Conformal: 보정(calibration) 집합에서 잔차(residual)의 분위수를 계산, 해당 분위수를 예측값에 더하고 빼서 구간 생성
- Conformalized Quantile Regression (CQR): 하한/상한을 별도로 학습한 Quantile Regressor에 Conformal 보정을 적용, 이상치가 많은 꼬리 분포에서 더 좁은 구간 달성

6.4 학습 데이터와 파이프라인

- 데이터: 2024년 1월~2025년 12월, 미국 내 항공편 약 120만 건
- 피처: 출발·도착 공항, 시간대, 요일, 기상(시정, 풍속, 강수), 항공사, 기종, 거리
- Train/Validation/Calibration/Test: 60/15/15/10
- 하이퍼파라미터 튜닝: Optuna 100 trials, TPE 알고리즘, 5-fold CV
- 모델 관리: MLflow로 실험 추적·모델 레지스트리

6.5 왜 MLflow인가

ML 프로젝트는 파라미터·데이터·코드 조합에 따라 성능이 달라지므로, 모든 실험을 재현 가능하게 추적해야 합니다. MLflow는 Experiment Tracking, Model Registry, Model Serving을 통합 제공하며, 오픈소스이고 Python/Airflow와 쉽게 통합됩니다.

7. 이상 탐지 모델

7.1 왜 Isolation Forest인가

이상 탐지(Anomaly Detection)는 라벨이 없는 비지도 학습 문제입니다. '이상'은 정의가 애매하고 데이터도 적기 때 문입니다. Isolation Forest는 다음 이유로 적합합니다:

- 라벨 불필요: 정상/이상 구분이 필요 없음
- 속도: 트리 기반이라 학습·예측이 빠름
- 원리 직관적: '이상한 점은 몇 번의 분할로 고립된다' → 점수가 낮음
- 고차원 대응: 피처가 많아도 잘 동작

7.2 Per-Phase 모델 — 왜 운항 단계별로 나눴는가

이륙 중인 항공기와 순항 중인 항공기는 정상 패턴이 완전히 다릅니다. 예컨대 이륙 중에는 고도가 급격히 상승하는 게 정상이지만, 순항 중에는 비정상입니다. 단일 모델로 학습하면 '이륙 중 급상승'을 이상으로 오인합니다. 이를 해결하기 위해 운항 단계를 7개로 나누고 각 단계별로 Isolation Forest를 별도 학습합니다:

1. Parked (주기)
2. Taxi (지상 이동)
3. Takeoff (이륙)
4. Initial Climb (초기 상승)
5. Cruise (순항)
6. Approach (접근)
7. Landing (착륙)

7.3 Silver-Label 학습 (Weak Supervision)

라벨이 전혀 없는 Cold Start 상황에서는 '약한(weak) 규칙'으로 초기 라벨을 생성합니다. 예: 'METAR 저시정 + 고도 이탈 > 500ft'이면 가능성 있는 이상 후보로 분류. 이 Silver Label로 Isolation Forest를 학습하고, 사람이 피드백하는 과정을 반복하여 Gold Label로 수렴합니다.

7.4 SHAP 기반 설명 생성

이상 점수만 제공하면 관제사가 '왜 이상인지' 모릅니다. SHAP으로 각 피처의 기여도를 계산하여 '고도가 정상 범위 에서 +600ft 이탈, 기여도 0.45'와 같이 설명합니다. 이는 다음 LLM의 입력이 되어 자연어 조언을 생성합니다.

8. LLM과 RAG

8.1 왜 Qwen2.5-7B인가

LLM 후보가 많습니다: GPT-4, Claude, Llama-3, Qwen 등. 선택 기준과 결과:

모델	라이선스	파라미터	다국어	선택 이유 / 탈락 이유
GPT-4	상용 API	-	★★★★★	비용 + 데이터 외부 전송 이슈
Claude-3	상용 API	-	★★★★★	동일
Llama-3-8B	MetaLlama License	8B	★★★	한국어 성능 제한적
Qwen2.5-7B	Apache-2.0	7B	★★★★★	한국어 강점 + 자유 라이선스

온프레미스 배포·한국어 지원·파인튜닝 자유도가 필수이므로 Qwen2.5-7B를 선택했습니다.

8.2 왜 QLoRA 파인튜닝인가

7B 모델을 전체 파인튜닝하려면 GPU 메모리 100GB 이상이 필요합니다. QLoRA(Quantized LoRA)는 다음 기법으로 단일 24GB GPU에서도 학습 가능하게 합니다:

- 4-bit NF4 양자화로 기본 가중치 메모리 75% 절감
- LoRA(Low-Rank Adaptation)로 작은 어댑터만 학습
- Paged Optimizer로 OOM 방지

본 프로젝트는 RTX 4090 24GB 단일 GPU로 6시간 학습했습니다.

8.3 왜 DPO인가

단순 SFT(Supervised Fine-Tuning)는 '옳은 답'만 학습합니다. 하지만 항공 도메인에서는 '더 안전한 답'이 중요합니다. DPO(Direct Preference Optimization)는 (선호되는 답, 덜 선호되는 답) 쌍으로 학습하여, 모델이 선호를 이해하도록 만듭니다. RLHF보다 구현이 간단하고 안정적입니다.

8.4 왜 RAG인가

LLM은 지식이 학습 시점에 고정됩니다. 항공 규정은 자주 업데이트되고, NOTAM은 실시간입니다. RAG(Retrieval-Augmented Generation)는 다음을 해결합니다:

- 최신 정보 반영: ChromaDB에 ICAO, FAA AIM, NOTAM을 벡터화하여 저장
- Hallucination 감소: LLM이 검색된 출처를 근거로 답변 → 거짓 정보 감소
- 출처 표시: 답변에 '출처: FAA AIM 5-1-2' 포함하여 신뢰성 확보

8.5 왜 BAI/bge-m3 임베딩인가

벡터 검색의 품질은 임베딩 모델이 좌우합니다. bge-m3는 다국어(100개 이상 언어)를 지원하고, Dense/Sparse/Multi-vector 3가지 검색 모드를 함께 제공하여 한국어 + 영어 혼재 항공 문서에 적합합니다.

8.6 왜 vLLM 서빙인가

LLM 추론은 GPU 메모리 대비 처리량이 문제입니다. vLLM은 PagedAttention(가상 메모리 기법 차용)으로 KV 캐시를 효율적으로 관리하여 기존 HuggingFace Transformers 대비 10~24배 빠른 처리량을 달성합니다. AWQ 4-bit 양자화와 결합하면 7B 모델이 16GB VRAM에서도 동작합니다.

9. 서빙 API와 대시보드

9.1 왜 FastAPI인가

Python 웹 프레임워크는 Flask, Django, FastAPI 등이 있습니다. FastAPI를 선택한 이유:

- Pydantic 기반 자동 검증: 요청·응답 스키마 런타임 체크
- OpenAPI 자동 생성: /docs에서 바로 Swagger UI 제공
- 비동기(async) 네이티브: 높은 처리량
- 타입 힌트 기반: IDE 자동완성과 정적 분석이 우수

9.2 15개 엔드포인트 요약

엔드포인트	용도
POST /predict/delay	지연 예측 + 신뢰구간
POST /predict/anomaly	이상 점수 + SHAP 설명
POST /chat	LLM 조언 생성 (RAG)
GET /stream/positions	SSE 실시간 위치 스트림
GET /stream/anomalies	SSE 이상 알림 스트림
WS /ws/events	WebSocket 실시간 이벤트
GET /active-learning/next	불확실성 기반 라벨링 우선순위
POST /feedback	사람 피드백 수집
GET /models/active	현재 활성 모델 버전
POST /models/rollback	이전 모델로 롤백
GET /health/live	Liveness 프로브
GET /health/ready	Readiness 프로브
GET /metrics	Prometheus 메트릭
POST /admin/train	재학습 트리거 (HITL 승인)
GET /lineage/trace	OpenLineage 추적

9.3 왜 Next.js 16 App Router인가

대시보드는 실시간 데이터를 표시해야 하므로 SSR(Server-Side Rendering)과 CSR(Client-Side Rendering)을 적절히 섞어야 합니다. Next.js 16 App Router는 서버 컴포넌트·스트리밍·Suspense-rewrites(프록시)를 네이티브 지원하여 이에 적합합니다.

9.4 Rewrites 기반 Same-Origin 프록시

브라우저에서 http://api:8000을 직접 호출하면 Docker 내부 DNS를 해석하지 못합니다. Next.js rewrites로 /api/*를 내부 API 컨테이너로 프록시하여 브라우저는 항상 자기 도메인만 호출합니다. 이는 CORS 이슈도 해결합니다.

10. MLOps와 관측성

10.1 왜 관측성이 중요한가

ML 시스템은 '코드가 맞게 동작하는가'뿐 아니라 '모델 성능이 시간에 따라 어떻게 변하는가'도 감시해야 합니다. 데이터 분포가 바뀌면(Data Drift) 모델 성능이 서서히 떨어지고, 이를 모르면 잘못된 예측이 운영에 스며듭니다.

10.2 OpenTelemetry — 왜 필요한가

분산 시스템에서 요청 하나가 Kafka → Flink → ML → LLM → 대시보드로 흐릅니다. 어디서 느려졌는지 모르면 디버깅이 불가능합니다. OpenTelemetry는 모든 서비스에 Trace ID를 심어 전체 경로를 Jaeger에서 시각화할 수 있게 합니다.

10.3 Prometheus + Grafana

- Prometheus: 메트릭 수집 (CPU, Latency, 처리량, 모델 점수 분포)
- Grafana: 대시보드 시각화, Alert 규칙 (SLO 위반 시 알림)
- Alertmanager: Slack/PagerDuty로 통보

10.4 Data/Model Drift 감지 — EvidentlyAI

EvidentlyAI는 학습 데이터와 실시간 데이터의 분포 차이(KS test, PSI)를 자동 계산하여 Drift Report를 생성합니다. Grafana에 연동되어 이상 시 Alert가 발생합니다.

10.5 OpenLineage — Data Lineage

어떤 모델이 어떤 데이터로 학습되었는지, 그 데이터가 어디서 왔는지를 자동 추적합니다. 사고 시 '어느 단계에 문제가 있었나'를 5분 안에 파악할 수 있습니다.

10.6 왜 Feast (Feature Store)인가

학습에 쓴 피처와 서빙에 쓰는 피처가 다르면 모델이 맞지 않습니다(Training-Serving Skew). Feast는 피처 정의를 중앙화하여 학습과 서빙이 같은 피처를 쓰도록 보장합니다.

10.7 Active Learning — 인간 피드백

모델이 확신하지 못하는 샘플(이상 점수가 임계치 근처)을 사람에게 먼저 라벨링 요청하여, 적은 라벨로도 큰 성능 향상을 달성합니다. /active-learning/next 엔드포인트가 Uncertainty Sampling으로 우선순위를 부여합니다.

11. 배포 전략

11.1 왜 Docker Compose로 시작했는가

개발 환경에서 7개 컨테이너를 쉽게 띄우려면 docker-compose가 가장 단순합니다. 본 프로젝트는 Synology NAS(저사양)에 배포해 실제 캠퍼스 네트워크 외부에서 접근 가능하게 했습니다.

11.2 왜 Helm + Kubernetes인가 (운영)

여러 환경(dev/staging/prod)에 같은 애플리케이션을 약간씩 다른 설정으로 배포해야 합니다. Helm Chart는 values.yaml로 환경별 오버라이드를 제공합니다. Kubernetes는 자동 복구·수평 확장·롤링 업데이트를 기본 지원합니다.

11.3 왜 Terraform + Argo Rollouts인가

Terraform으로 인프라(VPC, Kubernetes 클러스터, DB)를 코드로 정의하여 수작업 실수를 없앱니다. Argo Rollouts는 Canary 배포와 자동 롤백을 지원하여, 새 버전을 5% 트래픽으로 먼저 테스트 후 문제 없으면 100%로 확장합니다.

11.4 보안 — 왜 Sigstore cosign과 SBOM인가

- cosign: 컨테이너 이미지에 서명 → 공급망 공격(Supply Chain Attack) 방어
- SBOM (Software Bill of Materials): syft로 SPDX/CycloneDX 포맷 생성 → 의존성 투명성
- Trivy 스캔: 이미지 내 CVE 자동 탐지

11.5 왜 GitHub Actions CI/CD인가

GitHub과 통합이 자연스럽고, 오픈소스에 무료 크레딧이 충분합니다. 본 프로젝트는 PR 시 lint·test·build·scan을 자동화하고, tag 푸시 시 GHCR에 이미지를 빌드·서명·publish하는 파이프라인을 구성했습니다.

12. 성능 평가

12.1 지연 예측 모델

메트릭	값	의미
MAE	7.8분	평균 절대 오차
RMSE	12.3분	큰 오차 벌점
R ²	0.62	설명력
Conformal Coverage ($\alpha=0.1$)	89.7%	목표 90% 근접
CQR Avg Width	23.4분	구간 너비

12.2 이상 탐지

모델	Precision	Recall	F1
Single IF (baseline)	0.72	0.68	0.70
Per-Phase IF × 7	0.88	0.81	0.84

12.3 LLM 평가 (RAGAS)

메트릭	값
Faithfulness	0.91
Answer Relevancy	0.88
Context Precision	0.85
Context Recall	0.82

12.4 서버 지연

엔드포인트	P50	P95	P99
/predict/delay	18ms	45ms	78ms
/predict/anomaly	22ms	58ms	96ms
/chat (vLLM)	1.2s	2.8s	4.5s

13. 결론 및 향후 과제

13.1 달성한 것

- 엔드투엔드 항공 AI 플랫폼 구축 (데이터 → ML → LLM → 서빙 → 관측)
- 안전·해석 가능성을 고려한 모델 선택 (XGBoost + Conformal, Per-Phase IF, Qwen + RAG)
- 엔터프라이즈급 MLOps/LLMOps 파이프라인 (MLflow, Airflow, EvidentlyAI, Feast)
- 컨테이너·오케스트레이션·IaC 기반 배포 자동화 (Docker, Helm, Terraform, Argo)
- 공급망 보안 (cosign, SBOM, Trivy)

13.2 한계와 개선점

- 실제 운항 환경 테스트 부족 — 시뮬레이션 데이터 중심
- LLM 응답 지연 (P95 2.8s) — Speculative Decoding 도입 필요
- Per-phase 라벨링 비용 — Active Learning으로 완화 중
- 다중 지역(Multi-Region) 배포 미구현

13.3 향후 과제

1. 실제 항공사 파트너십으로 운영 데이터 피드백 루프 구축
2. LLM 에이전트화 — Tool Use로 자동 액션 (예: NOTAM 자동 발행 요청)
3. A-CDM(Airport Collaborative Decision Making) 표준 데이터 통합
4. Federated Learning — 여러 공항의 모델을 프라이버시 보존하며 결합
5. Edge 배포 — Airbus/Boeing 항공기 내 경량 모델 추론

14. 참고문헌

- Chen & Guestrin. XGBoost: A Scalable Tree Boosting System. KDD 2016.
- Lundberg & Lee. A Unified Approach to Interpreting Model Predictions. NeurIPS 2017.
- Liu, Ting & Zhou. Isolation Forest. ICDM 2008.
- Angelopoulos & Bates. A Gentle Introduction to Conformal Prediction. 2021.
- Romano, Patterson & Candès. Conformalized Quantile Regression. NeurIPS 2019.
- Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023.
- Rafailov et al. Direct Preference Optimization. NeurIPS 2023.
- Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- Kwon et al. Efficient Memory Management for LLM Serving with PagedAttention (vLLM). SOSP 2023.
- FAA. Aeronautical Information Manual (AIM). 2025 Edition.
- ICAO. Annex 2 — Rules of the Air. 11th Edition.
- Armbrust et al. Lakehouse: A New Generation of Open Platforms. CIDR 2021.

부록 A · API 엔드포인트 명세

모든 엔드포인트는 /docs(Swagger UI)에서 대화식으로 테스트 가능합니다. 아래는 대표 3개의 상세 명세입니다.

A.1 POST /predict/delay

```
Request:
{
  "origin": "KSFO",
  "destination": "KJFK",
  "scheduled_departure": "2026-04-16T14:00:00Z",
  "carrier": "UAL",
  "aircraft_type": "B738",
  "weather_visibility": 10.0,
  "weather_wind_kt": 12
}

Response 200:
{
  "predicted_delay_min": 14.8,
  "conformal_interval": {
    "lower": 9.2,
    "upper": 22.1,
    "confidence": 0.90,
    "method": "cqr"
  },
  "top_factors": [
    {"feature": "weather_visibility", "shap": -0.32},
    {"feature": "origin_congestion", "shap": 0.28}
  ],
  "model_version": "v2.1.3",
  "trace_id": "7a3f..."
}
```

A.2 POST /predict/anomaly

```
Request:
{
  "flight_id": "UAL123",
  "phase": "cruise",
  "features": { ... }
}

Response 200:
{
  "anomaly_score": 0.87,
  "severity": "HIGH",
  "phase_model_used": "cruise_if_v4",
  "shap_reasons": [
    {"feature": "altitude_delta", "contribution": 0.41, "description": "순항 고도 +620ft 이탈"}
  ],
  "suggested_action": "조언이 필요하면 POST /chat를 호출하세요."
}
```

A.3 POST /chat

Request:

```
{
  "alert_id": "anom-20260416-1458-UAL123",
  "question": "이 이상 상황에 어떻게 대처해야 하나요?"
}
```

Response 200:

```
{
  "answer": "고도 이탈이 감지되었습니다. FAA AIM 5-3-5에 따라 ATC와 고도 확인 후 ...",
  "citations": [
    {"source": "FAA AIM 5-3-5", "score": 0.91},
    {"source": "ICAO Annex 2 §3.1", "score": 0.87}
  ],
  "tokens_used": 420,
  "latency_ms": 1180
}
```

부록 B · 배포 체크리스트

B.1 개발 환경

- docker-compose up -d 로 전체 스택 기동 확인
- make seed 로 샘플 데이터 주입
- curl http://localhost:8000/health/ready → 200 OK
- http://localhost:3000 대시보드 접속 확인

B.2 스테이징 배포

- helm upgrade --install skyops ./charts/skyops -f values-staging.yaml
- kubectl rollout status deploy/api -n staging
- OTel/Grafana에서 SLO 그래프 녹색 유지
- synthetic smoke test 통과 (pytest -m smoke)

B.3 운영 배포 (Canary)

- Argo Rollouts가 신규 버전에 5% 트래픽 할당
- 5분간 P95 Latency, Error Rate 모니터링
- 정상이면 25% → 50% → 100% 자동 증분
- SLO 위반 시 자동 롤백

B.4 사고 대응 런북 (Runbook)

1. Grafana Alert 확인 → 어떤 SLO 위반인지 파악
2. Jaeger Trace로 병목 서비스 식별
3. OpenLineage로 데이터 파이프라인 추적
4. kubectl logs로 최근 에러 수집
5. 필요 시 argo rollouts abort 후 롤백
6. 포스트모템 작성 후 runbook 업데이트

부록 C · 용어집

용어	설명
ADS-B	Automatic Dependent Surveillance-Broadcast: 항공기 위치 자동 방송
METAR	Meteorological Aerodrome Report: 공항 정기 기상 관측 보고
NOTAM	Notice to Airmen: 비행 안전 관련 공지
SWIM	System Wide Information Management: FAA 공유 정보 시스템
AIXM	Aeronautical Information Exchange Model: 항공 정보 교환 표준
A-CDM	Airport Collaborative Decision Making: 공항 협업 의사결정
ATFM	Air Traffic Flow Management: 항공 교통 흐름 관리
SHAP	SHapley Additive exPlanations: 모델 설명 기법
Conformal Prediction	분포 가정 없이 신뢰구간을 보장하는 통계 기법
QLoRA	Quantized Low-Rank Adaptation: 양자화 기반 경량 파인튜닝
DPO	Direct Preference Optimization: 선호 기반 LLM 정렬
RAG	Retrieval-Augmented Generation: 검색 증강 생성
PagedAttention	vLLM의 KV 캐시 관리 기법 (가상 메모리 차용)
OpenTelemetry	분산 추적·메트릭·로그의 오픈 표준
OpenLineage	데이터 리니지 추적 오픈 표준
SLO	Service Level Objective: 서비스 품질 목표
SBOM	Software Bill of Materials: 소프트웨어 의존성 목록
Helm Chart	Kubernetes 애플리케이션 패키징 도구
Argo Rollouts	Kubernetes Progressive Delivery 컨트롤러
HITL	Human-in-the-Loop: 사람 개입 승인 단계

본 보고서는 2026-04-16 자동 생성되었으며, SkyOps Intelligence v2.1.3을 기준으로 합니다.